EXP 5	Programming exercises on maximizing a function using Genetic algorithm.

AIM: Understand the concept of maximizing function using Genetic algorithm using python programming.

Algorithm:
1. Initialize the population with random solutions.
2. Define the fitness function to evaluate how good each solution is.
3. Set the maximum number of generations.
4. Set the mutation rate (probability of changing a gene in an individual).
5. Set the crossover rate (probability of two individuals mating).
6. Repeat for each generation:
   a. Evaluate the fitness of each individual in the population using the fitness function.
   b. Select the best individuals based on their fitness to become parents.
   c. Create a new generation by crossover (mixing) the genes of the parents.
   d. Apply mutation to some individuals in the new generation.
   e. Replace the old population with the new generation.
7. Repeat for the specified number of generations.
8. Find and return the individual with the highest fitness as the best solution.

PROGRAM:

```
import random

# Define the fitness function (our objective function to maximize)
def fitness_function(x):
    return -x**2 + 6*x + 9

# Initialize the population
def initialize_population(pop_size, lower_bound, upper_bound):
    return [random.uniform(lower_bound, upper_bound) for _ in range(pop_size)]

# Select parents based on their fitness
def select_parents(population):
    total_fitness = sum(fitness_function(individual) for individual in population)
    roulette_wheel = [fitness_function(individual) / total_fitness for individual in population]
    parent1 = random.choices(population, weights=roulette_wheel)[0]
    parent2 = random.choices(population, weights=roulette_wheel)[0]
    return parent1, parent2

# Perform crossover to create a new generation
def crossover(parent1, parent2, crossover_prob=0.7):
    if random.random() < crossover_prob:
        crossover_point = random.randint(1, 1)  # Corrected this line
        child1 = (parent1 + parent2) / 2
        child2 = (parent1 + parent2) / 2
        return child1, child2
    else:
```

```
        return parent1, parent2

# Perform mutation in the population
def mutate(individual, mutation_prob=0.01):
    if random.random() < mutation_prob:
        individual += random.uniform(-1, 1)
    return individual

# Genetic Algorithm
def genetic_algorithm(generations, pop_size, lower_bound, upper_bound):
    population = initialize_population(pop_size, lower_bound, upper_bound)

    for gen in range(generations):
        new_population = []

        while len(new_population) < pop_size:
            parent1, parent2 = select_parents(population)
            child1, child2 = crossover(parent1, parent2)
            child1 = mutate(child1)
            child2 = mutate(child2)
            new_population.extend([child1, child2])

        population = new_population

        best_individual = max(population, key=fitness_function)
        print(f"Generation {gen+1}: Best individual - {best_individual}, Fitness -
{fitness_function(best_individual)}")

    return max(population, key=fitness_function)

if __name___== "__main__":
    generations = 20
    pop_size = 100
    lower_bound = -10
    upper_bound = 10

    best_solution = genetic_algorithm(generations, pop_size, lower_bound, upper_bound)
    print(f"Best solution found: {best_solution}, Fitness: {fitness_function(best_solution)}")
```

Output:

```
Generation 1: Best individual - 3.6731766347831503, Fitness - 17.546833218382034
Generation 2: Best individual - -5.767816408334088, Fitness - -58.87460457025247
Generation 3: Best individual - -6.771024276552948, Fitness - -77.47291541298706
Generation 4: Best individual - -7.225059724630964, Fitness - -86.55184637227025
Generation 5: Best individual - -7.382084793216393, Fitness - -89.78768465353508
Generation 6: Best individual - -7.442608835863151, Fitness - -91.04807929884716
Generation 7: Best individual - -7.442608835863151, Fitness - -91.04807929884716
```

```
Generation 8: Best individual - -7.48016970915957, Fitness - -91.8339571327858
Generation 9: Best individual - -7.48016970915957, Fitness - -91.8339571327858
Generation 10: Best individual - -7.731908615465194, Fitness - -97.17386253069606
Generation 11: Best individual - -7.8603227454545195, Fitness - -99.9466101354368
Generation 12: Best individual - -7.8603227454545195, Fitness - -99.9466101354368
Generation 13: Best individual - -7.8603227454545195, Fitness - -99.9466101354368
Generation 14: Best individual - -7.967982498887742, Fitness - -102.29664009590779
Generation 15: Best individual - -7.981046927325184, Fitness - -102.58339162011785
Generation 16: Best individual - -7.539224248089075, Fitness - -93.07524775150873
Generation 17: Best individual - -7.797894666671159, Fitness - -98.59452923252545
Generation 18: Best individual - -7.807113264577051, Fitness - -98.79369711339726
Generation 19: Best individual - -7.807113264577051, Fitness - -98.79369711339726
Generation 20: Best individual - -8.019944515886834, Fitness - -103.43917713322432
Best solution found: -8.019944515886834, Fitness: -103.43917713322432
```

Result: Thus the above program maximizing function using genetic algorithm executed successfully with desired output.